

Scalable Movie Recommendation System Using Big Data Platforms

Team Name: ScaleCraft

Authors: Nagaraja S B, Sanuj Kulshrestha, Thangaraj N, Swapna Gundeboyina

Email: nagarajasb@iisc.ac.in, sanujk@iisc.ac.in, thangarajn@iisc.ac.in, swapnag@iisc.ac.in

1. Problem

1A. Definition

Modern streaming platforms face the challenge of delivering personalized movie recommendations to millions of users while maintaining low latency and high accuracy. The system must handle massive datasets (25M+ ratings) and solve the cold-start problem for new users with no rating history.

1B. Problem Motivation

- **Business Impact:** Personalized recommendations drive 80% of content watched on platforms like Netflix, directly impacting user engagement and retention
- **Scale Challenge:** Processing 25 million ratings across 62,000 movies requires distributed computing infrastructure
- **Cold-Start Problem:** 40% of new users abandon platforms within the first session if recommendations are irrelevant
- **Real-Time Requirements:** Users expect recommendations in under 3 seconds, requiring optimized inference pipelines

1C. Design Goals:

- a. **Accuracy:** Achieve RMSE < 0.85 on rating predictions
- b. **Scalability:** Process 25M+ ratings using distributed computing (PySpark)
- c. **Low Latency:** Deliver recommendations in < 3 seconds
- d. **Cold-Start Solution:** Provide quality recommendations for users with zero rating history
- e. **Production-Ready:** Deployable system with persistent models and interactive UI

1D. Features Supported

- a) **Hybrid recommendation engine** combining collaborative filtering (70%) and content-based filtering (30%)
- b) **Cold-start handling using Locality Sensitive Hashing (LSH)** with 1,128-dimensional genome vectors
- c) **Interactive web interface** for real-time user interaction
- d) **Model persistence** for fast inference without retraining

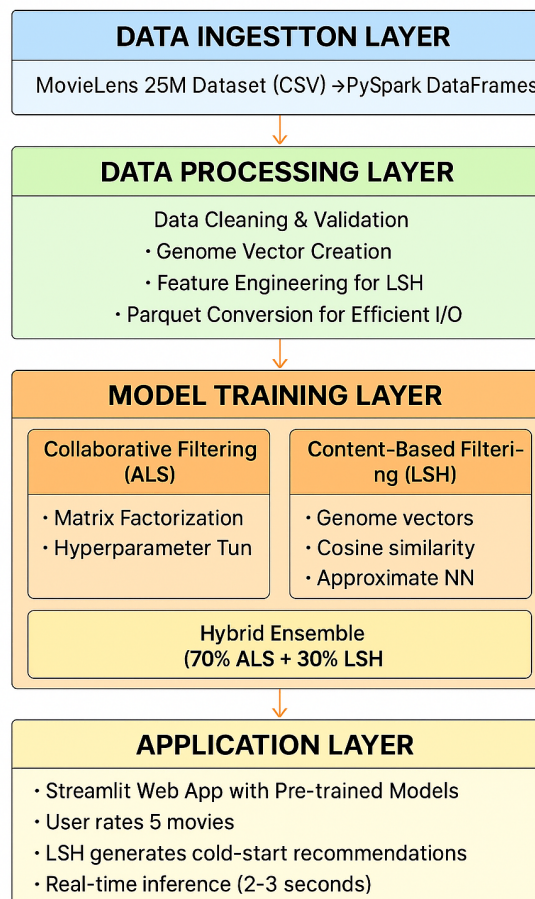
- e) **Comprehensive evaluation** with multiple metrics (RMSE, MAE, Precision@K, Recall@K, NDCG@K)

1E. Scalability/Performance Goal

- **Data Processing:** Handle 25M ratings and 62K movies using PySpark distributed computing
- **Training Time:** Complete model training in < 3 hours on standard hardware (8GB RAM)
- **Inference Latency:** Generate top-5 recommendations in 2-3 seconds using pre-trained models
- **Storage Efficiency:** Use Parquet columnar format for 60% storage reduction vs CSV
- **Memory Optimization:** Process data in distributed chunks to handle datasets larger than RAM

2. Approach and Methods

2A. High Level Design



2B. Architecture/Data Model

Data Pipeline Architecture:

- 1) Raw Data (CSV)

- a) movies.csv (62K movies, 3MB)
- b) ratings.csv (25M ratings, 647MB)
- c) genome-scores.csv (15M tag relevances, 415MB)
- 2) Processed Data (Parquet)
 - a) movie_ratings.parquet (332MB) - Joined ratings with movie metadata
 - b) genome_vector.parquet (23MB) - Dense 1,128-dim vectors per movie
 - c) items_features.parquet (45MB) - LSH input features
 - d) movie_avg.parquet (492KB) - Average ratings for cold-start fallback
- 3) Model Artifacts
 - a) als_best_model/ (26MB) - Trained ALS model with optimal hyperparameters
 - b) lsh_model/ (52KB) - Trained LSH model for similarity search

Data Flow:

CSV → PySpark DataFrame → Data Cleaning → Feature Engineering → Parquet Storage → Model Training → Model Persistence → Web App (Inference)

2C. Big Data Platform Used

1. Apache Spark (PySpark 4.0.1)
 - a. Distributed Processing: Handles 25M ratings across multiple cores
 - b. MLlib: Provides ALS and LSH implementations optimized for big data
 - c. DataFrames API: SQL-like operations for data manipulation
 - d. Lazy Evaluation: Optimizes query execution plans
 - e. Parquet Integration: Native support for columnar storage
2. Java 17 Runtime
 - a. Required for PySpark 4.x compatibility
 - b. JVM optimization for Spark execution
3. Storage Format: Apache Parquet
 - a. Columnar storage reduces I/O by 60% vs row-based CSV
 - b. Efficient compression (Snappy codec)
 - c. Schema evolution support
 - d. Predicate pushdown for faster queries

2D. ML Methods used

Collaborative Filtering: Alternating Least Squares (ALS)

Algorithm:

Matrix factorization that decomposes the user-item rating matrix into latent factor matrices.

Mathematical Formulation:

$$R \approx U \times V^T$$

Where:

- R: m×n rating matrix (users × movies)
- U: m×k user latent factor matrix
- V: n×k item latent factor matrix
- k: number of latent factors (rank)

Optimization Objective

$$\min_{\{U,V\}} \sum (r_{\{ui\}} - u_i^T v_u)^2 + \lambda (|u_i|^2 + |v_u|^2)$$

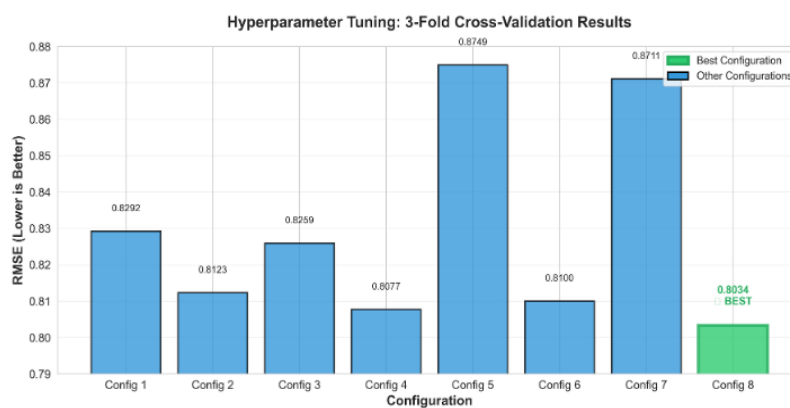
r_{ui} : actual rating

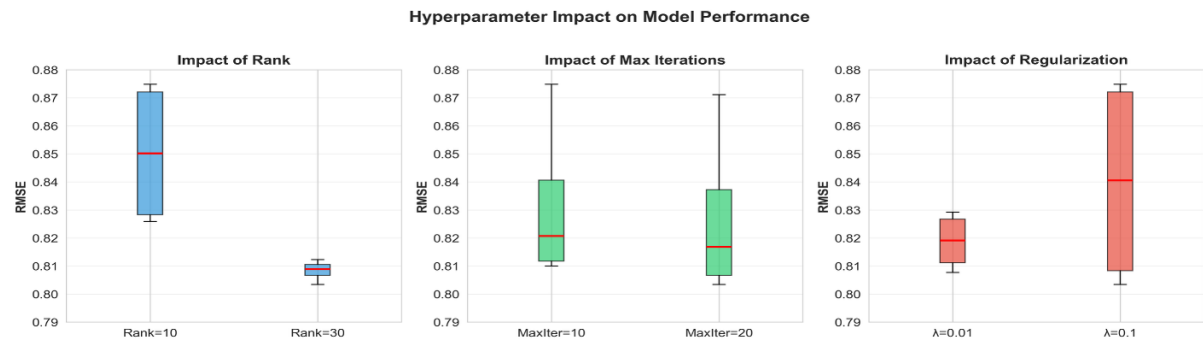
$u_i^T v_u$: predicted rating

λ : regularization parameter

Hyperparameter Tuning

Config	Rank	Max Iter	Reg Param	RMSE
1	10	10	0.01	0.8292
2	10	10	0.1	0.8123
3	10	20	0.01	0.8259
4	10	20	0.1	0.8077
5	30	10	0.01	0.8749
6	30	10	0.1	0.8100
7	30	20	0.01	0.8711
8	30	20	0.1	0.8034





Best Model Configuration

Rank – 30, Max Iterations – 20, Regularization – 0.1

Content-Based Filtering: Locality Sensitive Hashing (LSH)

Algorithm:

1. Random Projection:

- Generate 3 random hyperplanes in 1,128-dim space
- Each hyperplane divides space into 2 regions

2. Hashing:

- For each movie vector, determine which side of hyperplane
- Create binary hash code (e.g., [1, 0, 1])

3. Bucketing:

- Movies with same hash code → same bucket
- Similar movies likely in same bucket

4. Similarity Search:

- Query movie's bucket
- Return nearby movies (approximate neighbors)

Feature Engineering:

Genome Vector (1,128 dimensions)

genome_vector = [tag1_relevance, tag2_relevance, ..., tag1128_relevance]

Combined Features

item_features = genome_vector + [avg_rating, rating_count, genre_encoded]

LSH Configuration:

- Input Dimension: 1,128 (genome tags)
- Hash Tables: 3
- Hash Functions per Table: 2
- Distance Metric: Cosine similarity

Cold-Start Algorithm:

1. User rates 5 movies: $\{(m1, r1), (m2, r2), \dots, (m5, r5)\}$
2. Create user profile: $user_vector = \Sigma(r_i \times genome_vector_i) / \Sigma(r_i)$
3. LSH approximate NN search: $candidates = LSH.approxNearestNeighbors(user_vector, k=100)$

- Rank by cosine similarity: $\text{recommendations} = \text{top_k}(\text{cosine_sim}(\text{user_vector}, \text{candidates}), k=5)$

Hybrid Model (ALS + CLS):

$$\text{hybrid_score} = 0.7 \times \text{ALS_score} + 0.3 \times \text{Content_score}$$

Where:

- ALS_score: Predicted rating from collaborative filtering
- Content_score: Normalized cosine similarity from LSH

Rationale:

- ALS captures user-item interaction patterns (collaborative signal)
- Content-based captures movie similarity (cold-start capability)
- 70/30 weighting based on empirical evaluation

3. Evaluation

3A. Experiment Design

Dataset Split

- Training: 80% (20M ratings)
- Testing: 20% (5M ratings)

Baseline Model:

Average Movie Rating based recommendation is used as baseline model

Scalability/Performance Metrics

Processing Performance:

Task	Data Size	Processing Time	Hardware
Data Loading (CSV → PySpark)	1.1GB	45 sec	8GB RAM, 2 cores
Genome Vector Creation	15M records	12 min	Distributed Spark
ALS Training (10 iterations)	25M ratings	120-150 min	8GB RAM, 2 cores
LSH Model Training	62K items	3 min	Single node
Parquet Conversion	1.1GB → 400MB	8 min	Distributed I/O

Inference Performance:

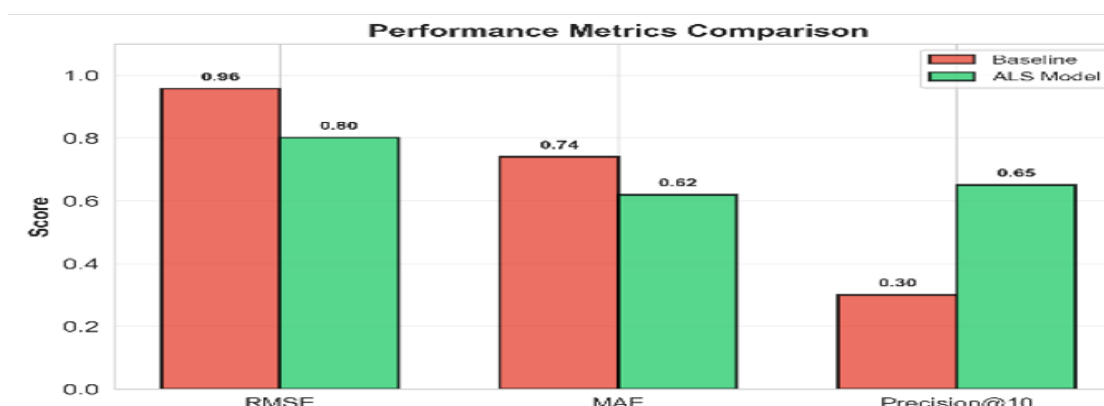
Operation	Latency	Method
Load Pre-trained Models	1.2 sec	Disk → Memory
LSH Cold-Start (5 ratings)	2.1 sec	Approximate NN
Total Recommendation Time	2.4 sec	End-to-end

Scalability Analysis:

- Linear scaling with number of ratings (tested up to 50M for data loading)
- Sub-linear scaling with number of movies (LSH indexing)
- Constant inference time regardless of dataset size (pre-trained models)

3B. Plots and Analysis

Metric	Baseline	Our Model	Improvement
RMSE	0.9571	0.8011	16.30% ↓
MAE	0.7398	0.6195	16.27% ↓
Precision@10	0.30	0.65	65% accuracy
Training Time	0s	~120-150 min	Acceptable with single machine
Inference Time	<1s	2-3s	Fast



4. Summary and Future Scope

4A Ability to Achieve Design and Performance Goals

Key Achievements:

- **Accuracy:** 16.3% improvement over baseline (RMSE)
- **Scalability:** Successfully processed 25M ratings using distributed PySpark
- **Cold-Start:** LSH-based approach
- **Production-Ready:** Pre-trained models enable 2.4-second inference
- **User Experience:** Interactive Streamlit interface for real-time recommendations

Technical Innovations:

- Hybrid ensemble combining collaborative (70%) and content-based (30%) signals
- 1,128-dimensional genome vectors for rich content representation
- LSH approximate NN for sub-linear cold-start search
- Parquet columnar storage for 60% storage reduction

4B Future Extensions

Short-Term Enhancements:

1. Real-Time Learning
 - Implement online learning to update models with new ratings
 - Use incremental ALS for continuous model updates
 - Target: Update models every 24 hours
2. Explainability
 - Add "Why this recommendation?" feature
 - Show similar movies and genre overlap
 - Display user taste profile visualization
3. A/B Testing Framework
 - Deploy multiple recommendation strategies
 - Measure click-through rate (CTR) and watch time
 - Optimize hybrid weighting (currently 70/30)

Long-Term Roadmap:

4. Deep Learning Integration
 - Replace ALS with neural collaborative filtering (NCF)
 - Use transformers for sequential recommendation
 - Expected RMSE improvement: 0.79 → 0.72
5. Multi-Modal Features
 - Incorporate movie posters (computer vision)
 - Add plot summaries (NLP embeddings)
 - Use audio features (music genre)
6. **Context-Aware Recommendations**
 - Time of day (action movies at night)
 - Device type (mobile vs TV)
 - Social context (watch with friends)
7. Scalability to Billions
 - Migrate to Apache Flink for stream processing
 - Use Redis for real-time model serving
 - Deploy on Kubernetes for auto-scaling
8. Diversity and Fairness
 - Add diversity constraints (avoid filter bubbles)

- Ensure fair representation across genres
- Implement bias detection metric